

Systems Performance Cheatsheet

L0 Essence: Systems performance diagnostics is guided by resource methodologies (such as the USE method), using hardware counters, kernel tracing hookpoints, and eBPF probes to perform non-invasive observation and bottleneck identification of CPU, memory, storage, and network utilization and saturation.

[M1.1] Three Core Performance Metrics: Latency, Throughput, and Efficiency

Performance tuning requires formal metric alignment: **Latency:** Also known as response time, the time elapsed between initiating an operation and its completion, measured in milliseconds or microseconds. Often follows a bi-modal or long-tailed distribution; focus should be placed on 99% or 99.99% tail latency. **Throughput:** The volume of work or number of transactions processed per unit of time (e.g., IOPS, QPS, Bytes/s). **Efficiency:** The resource consumption ratio required to process a given workload. Before throughput limits are reached, latency remains stable; once resources saturate, queuing effects kick in, latency rises exponentially, and efficiency drops.

[M1.2] Classic Resource Methodology: The USE Method

The USE method was created by Brendan Gregg for bottom-up, quick identification of physical hardware resource bottlenecks: **Scope:** Apply to all physical resources (CPUs, memory, disks, network interfaces, buses). **Utilization:** The percentage of time a resource is busy processing work over a given interval (e.g., CPU utilization at 80%), or the proportion of capacity used. **Saturation:** The degree to which a resource has queued work waiting to be processed (e.g., CPU run queue length, memory Swap-out rate). Saturation is the best leading indicator of performance bottlenecks. **Errors:** The absolute count of error events. When errors occur, system retries or error fallback loops degrade latency significantly.

[M1.3] Classic Application Methodology: The RED Method

The RED method was created by Tom Wilkie for top-down, service-oriented metrics in microservices and distributed systems: **Rate:** The number of requests processed per second (QPS/RPS), measuring the actual incoming workload. **Errors:** The number of failed requests per second (or error percentage), representing software application health. **Duration:** The time distribution taken to process requests (focusing on mean, p95, and p99 tails), representing end-user experience. **Complement:** USE focuses on physical resources; RED focuses on application logic. Together, they form full-stack monitoring.

[M1.4] Resource Saturation & Queueing Theory

System performance degradation is fundamentally a queueing theory problem caused by resource contention: **Queueing Model:** Resources can be modeled as $M/M/1$ or $M/M/c$ queues. As utilization (U) approaches 100%, average queue length and waiting time expand exponentially at a rate of $\frac{1}{1-U}$. **Knee Point:** Load testing aims to find the Knee Point on the throughput-latency curve. Before this point, adding concurrency raises throughput while latency remains flat; beyond it, throughput plateaus and latency spikes as the queue builds up, leading to congestion collapse.

[M1.5] Observer Effect & Performance Measurement Overhead

Every monitoring tool introduces some degree of disturbance to the system being observed: **Observer Effect:** The monitoring tool consumes CPU, competes for memory locks, or causes page faults, altering the original application behavior (which can mask or amplify bugs). **Instrumentation Overhead:** Dynamic probes require runtime instruction patching, causing CPU cache invalidations and branch instruction overhead; syscall tracing via **strace** uses the **ptrace** mechanism to pause the target process on every syscall, slowing it down by 10x to 100x. **Mitigation:** Limit sampling rates in production (e.g., **perf** at 99Hz or 997Hz to avoid resonance), use lockless ring buffers, and execute filters in kernel space to keep overhead under 1%.

[M2.1] CPU Hardware Performance: PMCs & NUMA Architecture

CPU tuning requires visibility into microarchitectural execution efficiency: **Performance Monitoring Counters (PMC):** Built-in CPU registers that count microarchitectural events (e.g., instructions per cycle (IPC), branch mispredictions, instruction cache misses, data cache misses). An IPC below 1.0 indicates the CPU is stalling, waiting for memory. **NUMA (Non-Uniform Memory Access):** Multi-socket systems partition CPU cores and memory. Accessing memory on a remote socket via QPI/UPI interconnects incurs multi-fold latency penalties. Core pinning (**numactl --physcpubind**) and local allocation strategies ensure NUMA node affinity.

[M2.2] CPU Scheduling & Load Averages

System scheduling queues and load statistics indicate CPU resource saturation: **Load Average:** Linux load averages count processes in **TASK_RUNNING** plus those in **TASK_UNINTERRUPTIBLE** (waiting on I/O), making load averages an imperfect indicator of pure CPU saturation. **Scheduling Latency:** The duration between a process being marked as runnable and actually being scheduled on a physical CPU core. Large run queues lengthen scheduling latency, leading to throughput drops.

[M2.3] CPU Profiling & Time-Based Stack Sampling

Locating hot code paths requires sampling-based profiling rather than instruction-level tracing: **Sampling Mechanism:** Profilers (e.g., **perf**) configure a hardware timer or OS timer interrupt at a set frequency (e.g., 99Hz). **Stack Capture:** The interrupt handler halts execution on the CPU, captures the Program Counter (PC), walks the stack frame pointers (or ORC debug info) backward, and records the full sequence of function calls. **Advantage:** Sampling does not intercept every function entry/exit, preserving flat, low overhead suitable for production environments.

[M2.4] Flame Graph Layout & Rendering Rules

Flame Graphs are a visualization technique created by Brendan Gregg for large-scale stack profiling: **Axes:** The Y-axis represents stack depth (base calling function at the bottom, leaf function at the top). The X-axis is not time; instead, all sampled stacks are sorted alphabetically to merge identical frames. **Width:** The width of each block represents the percentage of samples that matched that specific stack path. Wider blocks indicate functions that consume a larger portion of CPU time. **Color:** Warm palettes (Morandi red/yellow) are used. Colors are randomized only to distinguish neighboring blocks, with no numeric meaning.

L1 Logical Framework

Resource Methods & Metric Pinpointing: The USE method is used bottom-up to analyze hardware resources, coupled with the RED method top-down to dissect application performance, capturing saturation metrics to expose resource bottlenecks. **Scheduling Flames & Stack Profiling:** Stack traces are sampled periodically via timer interrupts, aggregating identical stack frames in Flame Graphs for visual hotness tracking to identify scheduling latency and lock contention bubbles. **Two-Level Page Faults & Virtual Memory:** Physical memory is managed at fine granularity via the Buddy System and Slab/Slub caches. Page faults are categorized into minor (mapping page table entries) and major (disk I/O blocked), and caching relies on Page Cache readahead and writeback. **IRQ Handling & eBPF Tracing:** Network packet reception and transmission flow through hard/soft IRQs and NAPI polling, while system diagnostics leverage the safe eBPF virtual machine and dynamic Kprobes/Tracepoints to capture kernel events with minimal overhead.

L2 Data Flow Topology

Application Request → RED Rate check → CPU instruction execute → PMC cache miss check → Run Queue queuing → Scheduling Latency → Memory access → TLB Hit? No → Page Table walk → Unmapped? Yes → Page Fault (Minor) → Mapped to Physical Page → Disk swap needed? Yes → Page Fault (Major) → VFS read() → Page Cache check → Cache Miss → Disk I/O Queue → I/O Scheduler (BFQ) → Disk read → iostat latency detection → Packet arrive → NIC Ring Buffer → Hard IRQ → Soft IRQ (ksoftirqd) → Socket Buffer Queue (rmem) → eBPF Kprobe hook → BPF Map update → bpftrace collect → Flame Graph rendering.

Trade-off Matrix: Diagnostics & Tracing

Dimension: Option A → Option B [Trade-off / Compromise]

Instrumentation Type: Static Tracepoints → Dynamic Kprobes / Up-robres [Tracepoints are precompiled with minimal overhead and do not break across kernel updates, but cannot trace arbitrary code. Dynamic probes hook any instruction but cause breknpoint trap overhead and can break on kernel structural changes. [Use static Tracepoints for production monitoring, and load dynamic Kprobes on demand for deep troubleshooting]]

CPU Profiling: Time-based Stack Sampling → Event-based Tracing [Sampling reads stacks at regular intervals (e.g., 99Hz) with low, predictable CPU overhead, though it can miss short-lived anomalies. Tracing hooks every function entry/exit, recording precise timings, but runs the risk of CPU interrupt storms on hot functions. [Deploy time-based stack sampling in production, and restrict event tracing to test environments]]

Network I/O Handling: Interrupt-driven Network I/O → Polling Mode (NAPI / PMD) [Interrupts sleep when idle, saving CPU cycles, but trigger interrupt storms under high traffic. Polling retrieves packets in batches to sustain gigabit networks but pegs CPU cores to 100% even when idle. [Use interrupt-driven I/O for low-bandwidth applications, and polling for high-speed routers or databases]]

Observability Resolution: Aggregated Profiling (Flame Graphs / USE) → System Call Tracing (strace) [Aggregated profiling summarizes metrics inside the kernel (e.g., BPF Maps), passing only summary charts to user space with minimal footprint. **strace** intercepts every syscall via ptrace, recording precise micro-order parameters but slowing execution down by 10x. [Do not run strace under load; compile Flame Graphs to isolate hot areas first]]

[M3.1] Virtual Memory Mapping: MMU, TLB, and Huge Pages

Virtual memory isolates address spaces but introduces translation costs: **MMU Page Table Walk:** The Memory Management Unit (MMU) walks multi-level page tables (4 or 5 levels on x86_64). Translating a virtual address requires 4 or 5 memory reads, which is expensive. **TLB Cache:** The Translation Lookaside Buffer (TLB) caches recently translated page table entries. A TLB miss forces page table walks, stalling the CPU. **Huge Pages:** Increasing page size from 4KB to 2MB or 1GB reduces page table depth and expands TLB coverage, avoiding TLB misses (commonly used in databases and JVM tuning).

[M3.2] Physical Memory Allocation: Buddy System & Slab/Slub

Physical memory frames are managed by a two-tiered allocation framework: **Buddy System:** Manages contiguous page blocks (4KB pages). Pages are tracked in buddy lists from Order-0 to Order-10 (4KB to 4MB). Requests split larger blocks; page releases merge neighboring blocks using XOR address calculations to prevent fragmentation. **Slab/Slub Allocator:** Buddy system operates on page levels. Fine-grained kernel objects (e.g., **task_struct**, **file**) use the Slab allocator, which slices pages from the Buddy system into small object slots, providing lockless thread-local caches (**kmem_cache**).

[M3.3] Page Reclaiming & Swap: Page Cache and Anonymous Pages

When physical memory falls below kernel watermarks, the reclaim process begins: `migrate_to_cache()`. Caches file copies in memory. Clean pages (unmodified) are discarded immediately with zero disk writeback cost. `migrate_to_swap()`: Anonymous pages (heaps, stacks, shared memory) cannot be discarded. Reclaiming them requires writing them to the disk swap partition, causing disk I/O blocks and millisecond-level latencies. `vm_swappiness`: Regulating `/proc/sys/vm/swappiness` (0 to 100) controls the kernel's preference for discarding Page Cache versus swapping anonymous pages.

[M3.4] Page Fault Diagnostics (Minor vs. Major Page Faults)

`mmap()` only reserves virtual address ranges; physical allocation is deferred to the page fault stage: `mmap_fault()`. Minor Page Fault: Triggered when a process reads or writes a virtual address that has no page table entry but already exists in memory (e.g., shared library pages or pages cached by other processes). The MMU simply adds the mapping, avoiding disk I/O (microsecond latency). Major Page Fault: Triggered if the page does not exist in memory. The kernel must fetch data from disk (e.g., loading file contents or reading swapped anonymous pages), causing blocking I/O and millisecond-level delays.

[M3.5] Memory Diagnostic Tools: vmstat, free, and OOM Killer

Memory saturations are monitored at different granularities: `vmstat`: Monitors global state. Metrics `si` (swap in) and `so` (swap out) record pages read/written per second. Consistent `so` above 0 indicates severe memory saturation. `free`: Checks overall free memory. Note `buff/cache` sizes and `available` (estimated memory that can be claimed without causing swap thrashing). `oomd`: OOM Killer: When physical RAM and swap are exhausted, the Out-of-Memory (OOM) killer selects a process to terminate based on memory consumption and `oom_score_adj`. The termination is logged in syslog.

[M4.1] Disk I/O Queuing and Service Latency

Disk tuning focuses on minimizing seek times and maximizing parallel queues: `hdparm`. HDD vs. SSD: HDDs are constrained by physical seek times (millisecond latency for random I/O); SSDs use flash memory, performing well on random reads/writes but suffering from write amplification. `iozone`: Equal to Queue Latency (time spent waiting in block queues) plus Service Latency (time taken by device firmware to read/write). High queue latency indicates disk saturation.

[M4.2] Virtual File System (VFS): Inodes and Dentries

VFS provides a unified interface and caches metadata in memory: `inode`: Contains file physical metadata (file size, timestamps, block pointers). Each file has a unique Inode number. `directory_entry` (Dentry): Maps paths to Inodes (e.g., `path test.txt` to Inode 1234). Dentries are cached in the Dentry Cache to accelerate path resolution, preventing disk accesses.

[M4.3] Caching: Page Cache and Writeback

The kernel uses memory buffer caches to offset slow disk speeds: `page_cache`: Caches file page data. Reads hit the cache directly; misses cause page faults, trigger `readahead` (prefetching adjacent blocks asynchronously), and load pages. `writeback`: Write operations update the Page Cache, mark the page as "dirty", and return immediately. The `writeback` kernel threads periodically flush dirty pages to disk asynchronously.

[M4.4] I/O Schedulers: Deadline, BFQ, and None

I/O schedulers sort and merge disk requests: `bfq` (Budget Fair Queueing): Allocates bandwidth budgets, optimizing for interactive desktop and media workloads at the expense of absolute throughput. `deadline`: Maintains separate read (500ms limit) and write (5s limit) queues. It prioritizes reads to prevent read starvation. `none` / Kyber: For fast NVMe SSDs. Since NVMe devices feature large parallel hardware queues, heavy kernel scheduling is bypassed (`none`) to avoid CPU overhead.

[M4.5] Disk I/O Diagnostic Tools: iostat, biolatency, and blktrace

Disk bottlenecks are diagnosed at three levels: `iostat`: Monitors global performance. Run `iostat -xz 1` to track `util` (percent of time busy); 100% does not imply saturation on SSDs due to parallel queues and `await` (read/write latencies, queue + service). `biolatency` (eBPF): Captures I/O latencies in a histogram instead of an average, highlighting tail latencies. `blktrace`: Logs every I/O event from entering queue (Q) to merger (M), driver issue (D), and device completion (C).

[M5.1] Network Data Paths: Hard IRQs, Soft IRQs, and NAPI

High-speed networks require optimized packet handling to avoid CPU context overheads: `hard_irq`: The network card receives packets, DMA-copies them to memory Ring Buffers, and signals a hardware interrupt. The CPU halts, acknowledges the interrupt, and triggers a Soft IRQ. `soft_irq` & NAPI: The `ksoftirqd` thread runs Soft IRQs. Using the NAPI framework, it disables hardware interrupts and polls the Ring Buffer, pulling packets in batches to prevent interrupt storms.

[M5.2] Socket Buffers & TCP Slide Windows

Network throughput is limited by socket queue sizes: `socket_buffers`: Each socket has a receive buffer (`rmem`) and a send buffer (`wmem`). If the receive buffer fills up because the application reads slowly, incoming packets are dropped. `bandwidth` Delay Product (BDP): The optimal buffer size equals $BDP = Bandwidth \times RTT$. `tcp_window` Alignment: TCP slide windows match buffer limits. The kernel auto-tunes `tcp_rmem` and `tcp_wmem` to match network RTT dynamically.

[M5.3] TCP Queue Overflows & BBR Congestion Control

Connection setup queues can cause transient connection timeouts: `half_open` (SYN) Queue: Holds connections in `SYN_RECV` state. Overflows are mitigated by `tcp_syncookies`. `accepted` Queue: Holds established connections waiting for `accept()`. Queue size is bounded by backlog and `/proc/sys/net/core/somaxconn`. If the application stalls, the queue overflows, causing ACK drops and retries. `bbrcntrl`: BBR Congestion Control: Unlike loss-based Cubic, BBR measures bottleneck bandwidth (RTTprop) and minimum RTT (BtlCwnd) to prevent bufferbloat, keeping delays low.

[M5.4] Network Diagnostic Tools: netstat, ss, and tcpdump

Network diagnostics require queue and socket polling tools: `ss`: Queries kernel netlink socket interfaces instead of parsing `/proc/net/tcp`. Run `ss -ltn` to read accept queue limits (`Send-Q`) and current backlogs (`Recv-Q`). `netstat`: Run `netstat -s` to track retransmissions and queue overflows. `tcpdump`: Performs packet capture. This incurs heavy libpcap copy overhead; use filters and restrict size (`-s` flag) in high-throughput environments.

[M6.1] Tracing Frameworks: Tracepoints and Kprobes

Linux supports multiple kernel instrumentation layers: `tracepoints`: Static tracepoints hardcoded by kernel developers at key hooks (e.g., `schedule`). They are stable and low overhead but restricted to predefined hooks. `kprobes` / `uprobes`: Dynamic probes that hook any kernel function (Kprobes) or user-space binary address (Uprobes) at runtime without restarts. `mechanism`: A Kprobe copies the target instruction and replaces it with an `int3` breakpoint. Reaching it triggers a trap handler executing the probe, then executes the original instruction.

[M6.2] eBPF Architecture: Virtual Machine, Verifier, and Maps

eBPF allows safe in-kernel execution without the risks associated with loading raw kernel modules: `virtual_machine`: A safe sandbox VM running 11 64-bit registers inside the kernel. `verifier`: Enforces safety by performing Directed Acyclic Graph (DAG) analysis on bytecode before loading, rejecting infinite loops, invalid pointers, and stack overflows. `bpf_maps`: Probes communicate with user space via maps (hash maps, arrays, ring buffers) to share metrics without copying raw trace files.

[M6.3] eBPF BCC Toolset Internals

BCC compiles, loads, and executes BPF probes in a single pipeline: `workflow`: BCC uses integrated Clang/LLVM compilation to compile BPF C code into bytecode at runtime. `execsnoop`: Hooks `sys_enter_execve` to trace new processes in real-time, bypassing slow `/proc` polls. `fileopsnoop`: Hooks `sys_enter_openat` to record all file opens across the system. `lifecycle`: Hooks TCP state transitions (e.g., `tcp_set_state`) to log RTTs, IP addresses, and lifespans.

[M6.4] bpftrace DSL Syntax and Maps

`bpftrace` is a DSL designed for rapid, on-the-fly tracing: `syntax`: Follows the type: `identifier:filter action`. E.g., `kprobe:vfs_read /comm == "nginx"/ @[comm] = count();` Hooks `vfs_read`, filters for "nginx", and counts instances. `maps` (`@`): Perform in-kernel aggregation (e.g., `hist()` maps latency to a logarithmic histogram), returning aggregated data to user space.

Zone T1: Performance Diagnostics CLI Commands

`perf record -F 99 -a -g --sleep 10` : Samples call stacks (-g) across all CPUs (-a) at 99Hz for 10 seconds, recording profiles to `perf.data`. `sysctl -w net.core.somaxconn=2048` : Expands the network accept queue limit to 2048 to prevent dropped connections under high load. `vmstat -SM 1` : Displays global resources in MB (-S M) every second, tracking run queues (r) and swap activities (si/so). `iostat -y -x -d 1` : Prints disk performance details every second, ignoring boot aggregations (-y) to trace active latencies (await).

Zone T2: bpftrace One-liners & Maps

`bpftrace -e 'tracepoint:syscalls:sys_enter_openat @[comm] = count();'` `bpftrace -e 'vfs_read() latency histogram: @[start[tid]/ @[latency] = hist(nsecs - @[start[tid]]); delete @[start[tid]];` `bpftrace -e 'tracepoint:syscalls:sys_enter_execve join(args->argv);'` `bpftrace -e 'tracepoint:exceptions:page_fault_user_minor @[comm] = count();'`